AI 编程实战训练营

把想法，交给 AI 做出来

第一讲 · 看懂软件世界，上手你的 AI 搭档 Codex



深度版 · 讲透“为什么” · 全程图形界面不碰命令行

今天这一讲

你会带走三样东西

1

看懂软件是怎么回事

前端后端、数据库、服务器、接口——以及一个应用怎么“跑起来”

2

上手你的第一个 AI 搭档

装好 Codex，并理解它的运作机制：Agent 循环、概率机器、审批

3

亲手做出一个网页

用一条好指令，让 Codex 从零做出能打开的作品，并学会验收

每一课都要动手，看懂不算学会——但今天我们也要把“为什么”讲透。

贯穿全程的两条原则

先记住这两句，后面反复回来

① 别信断言，亲自核实

App 说“做好了”、文档说“已完成”都不算数。你亲眼看到对的结果、或让它当场跑一遍验证，才算数。

② 先想清楚，再动手

最省时间的一步，永远是动手之前把“要什么、怎么算成”想明白。AI 执行极快，方向错了它会高效地把错的做完。

01

看懂软件世界

前端 · 后端 · 数据库 · 接口 · 服务器 · 请求响应

一个比喻

软件，就像一家餐厅

你看到的菜单和餐桌、

幕后做菜的后厨、

存食材的仓库、

传菜的服务员，

以及撑起这一切的餐厅楼。



记住这家餐厅，五个术语就都有了着落。

五个角色

餐厅 ↔ 软件，一一对应

前端	大堂 / 菜单 / 餐桌	你直接看到、点的部分
后端	后厨	幕后处理逻辑、真正“做菜”
数据库	食材仓库	存放和记住东西的地方
接口 API	服务员	在大堂和后厨之间传菜传单
服务器	整栋餐厅楼	让这一切 24 小时对外营业

这张表今天会一直用——每个术语都指回它对应的餐厅角色。

角色一 · 前端 Frontend

你直接看到、能点的一切

是什么：按钮、文字、图片、输入框、页面布局。

跑在哪：你自己的设备上（浏览器或 App）。

例子：你在淘宝看到的商品列表、点的“下单”按钮。

机制拆解 · 一句话分辨

眼睛看得到、手点得到的 = 前端。

你点了之后“幕后发生的事” = 后端（下一页）。

今天我们做的网页是纯前端——所有逻辑都在你浏览器里跑。

角色二 · 后端 Backend

藏在背后干活的部分

是什么：处理逻辑——下单后扣库存、算价格、生成订单。

跑在哪：远处的服务器上，你永远看不到它长什么样。

何时需要：要登录、要多人各自数据、数据不能丢时。

机制拆解 · 起步铁律

能纯前端解决的，就别急着上后端。

越简单越快跑通、越好维护。你前几课做的，都会是纯前端的小东西——这不是“低级”，是“恰当”。

前端 vs 后端

你看得见的 & 你看不见的

前端

按钮、文字、页面、输入框

你能看到、能点的一切

跑在你自己的设备上

后端

处理逻辑、算价格、连数据库

你点了之后“幕后发生的事”

跑在远处的服务器上

口诀：眼睛看得到、手点得到 = 前端；点了之后幕后的事 = 后端。

角色三 · 数据库 Database

让软件“记得住”

你的微信聊天记录、淘宝订单、账号密码，都存在数据库里。

存浏览器本地：
换台电脑、换个人，数据就没了。

存数据库（服务器上）：
换设备登录数据还在，每人各看各的。

机制拆解 · 何时才需要数据库

当“这个信息要长期留存、要跨设备、要多个人各看各的”时。

只给自己用、关了就不要的小工具，不需要数据库——这通常也意味着你还不需要后端。

角色四 · 接口 API

系统之间的“服务员”

接口是两段程序之间约定好的“对话方式”。前端按格式“下单”，后端按格式把结果送回——就像服务员和后厨的固定点单流程。

接口不只在前后端之间——不同公司的软件互相“喊话”也靠它：

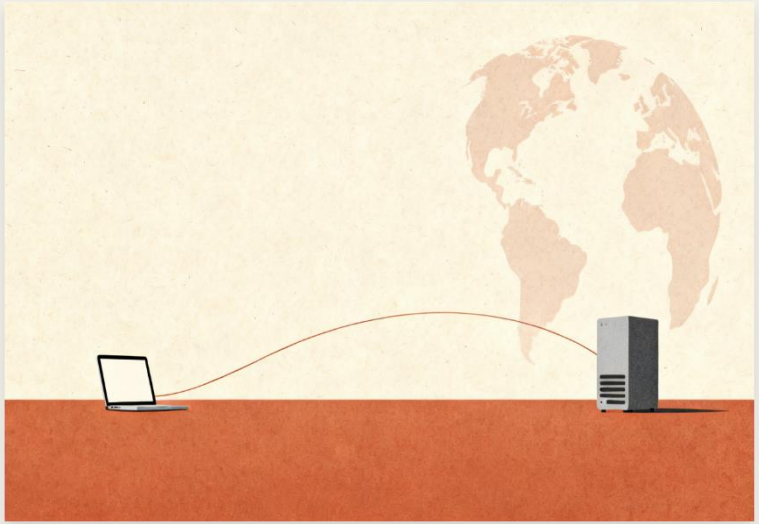
你在一个网站上用「微信登录」、看到嵌入的「高德地图」、完成「支付宝付款」——都是这个网站通过接口去调用了别家提供的能力。

以后听到“调个接口 / 对接某某的 API”，说的就是这件事。

角色五 · 服务器 Server

让软件 24 小时对外开着

服务器就是一台一直开着、连着网、专门对外提供服务的电脑。你的后端和数据库，就住在上面。



为什么必须有它?

你自己的电脑会关机、会断网、地址也不固定——别人没法随时访问你电脑里的东西。服务器 7×24 开着、有固定地址，全世界都能随时访问。

你的东西，现在“活”在哪？

本地

文件只在你这台电脑上。你双击能打开——但别人打不开。

服务器（上线）

放到一台一直开着的服务器上，全世界都能打开——这叫“部署”。

机制拆解 · 部署到底做了什么

① 把文件复制到托管平台的服务器；② 分配一个公开网址，并通过 DNS（互联网通讯录）把网址翻译成服务器地址，让任何人都能访问。

一次操作的全流程

点一次按钮，背后跑了一圈



例：淘宝点“加入购物车”。这一趟叫“请求 — 响应”，你在任何软件里的每一次操作，本质都是这么一圈。

机制拆解

为什么要“分层”？

因为分层让每部分能独立开发、独立替换：

换个前端界面，后端不用动；后端换个算法，前端无感；数据库搬个家，只要接口不变，其余照跑。

这也是为什么你和 AI 协作时，能只让它“改前端的按钮样式”而不动后端——它们本就是分开的。

决策框架

要不要后端 + 数据库?

自己用的小工具、展示页、计算器	→ **纯前端即可，甚至不用上线**
想把网页给别人看	→ **前端 + 部署到服务器（不需要后端）**
要登录、每人有自己的数据、数据不能丢	→ **前端 + 后端 + 数据库 + 服务器**
要用到别家能力（登录 / 地图 / 支付）	→ **再加 对接接口 API**

起步就问自己这张表——能纯前端解决的，绝不提前上后端。

02

上手 Codex

是什么 · Agent 循环 · 概率机器 · 网络 · 安装 · 第一次做网页

Codex 是什么

一个会“动手”的 AI 编程搭档

你说人话，**它直接在你电脑上**把文件建出来、把代码写好、把程序跑起来。

和聊天 AI 的区别只有两个字：动手。

聊天 AI 告诉你怎么做，Codex 替你做。



机制 · 它由两部分拼成

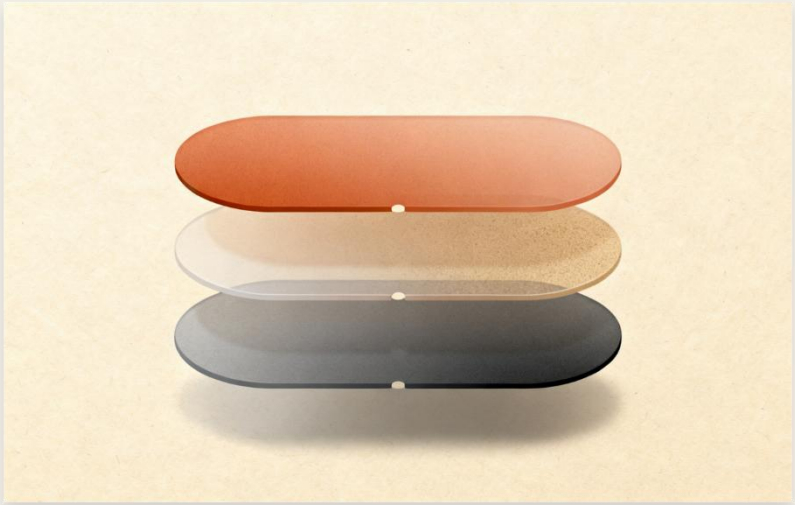
一个大脑 + 一套手脚

大脑 = 大语言模型（LLM）

学过海量代码与文字，本领是“预测最合理的下一段内容”。负责思考与生成。

手脚 = 代理外壳（Agent）

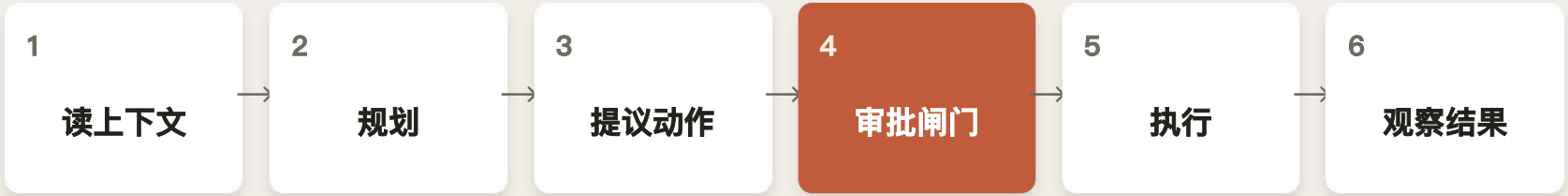
套在大脑外的程序，让它能真的读你的文件、改文件、跑命令、看运行结果，再喂回给大脑接着想。负责行动。



网页版 ChatGPT 大多只有大脑（只会说）；Codex 是大脑 + 手脚（能动手）。

机制 · 它怎么“动手”

Agent 循环：一步步来



↻ 执行完观察结果，回到第 ① 步，继续下一步

机制拆解 · 关键洞察

它每做一件实际的事（改文件、跑命令）之前，都要先“提议”——而审批和沙箱就卡在“提议”和“执行”之间那道闸门。这就是你能“先看预览再批准”的技术根源。

机制 · 它的底层性格

它是一台“概率预测机器”

大脑的目标是“生成看起来最合理的内容”，

而不是“说出经过核实的真相”。

它读过的东西太多，猜得极准——准到能写出能跑的代码。但当它没把握时，也会流畅、自信地编一个看起来对的答案。

机制 · 为什么会“幻觉”

这是原理决定的，躲不掉

想象一个语感极强的人，你问他一个他其实不知道的冷知识——出于语感，他能顺口编出一段“听起来非常像真的”答案。模型就是这样：它总能给出“最流畅合理”的续写，但流畅 ≠ 正确。

它甚至会说“我已经把文件建好了”——这也只是“预测出的合理措辞”，不代表它真做了、真对了。



所以第一条原则：别信断言，亲自核实。

决策框架

什么交给它 / 什么你把关

看什么	适合交给它	该你把关
能不能验收	做完对错一眼能判 （网页能不能打开）	好坏靠主观品味、说不清标准
可不可逆	错了能改、能还原	错了不可逆（删数据、对外发布）
要不要真实判断	有明确范例和套路	要拍板方向、要商业取舍

口诀：可验收、可撤销、有套路 → 放手交给它；要拍板、不可逆、凭品味 → 你来。

心智模型

你是指挥官，它是实习生

你： 定方向、把需求说清楚、验收对不对。

它： 读文件、写代码、跑程序，不知疲倦。

把它当一个很聪明、手很快、但刚来、不懂你们规矩、偶尔会自信记错事的实习生。
门槛不在会不会编程，在会不会指挥、验收、立规矩。



上手路径

三步，从零到能对话

1

打通网络

Codex 要连 OpenAI 的海外服务器
配好国际网络，浏览器自测

2

装好 App

像装微信一样：下载 → 拖进应用夹
登录 → 选一个练习文件夹

3

发第一条消息

收到它的回复
就代表安装/登录/连通全打通

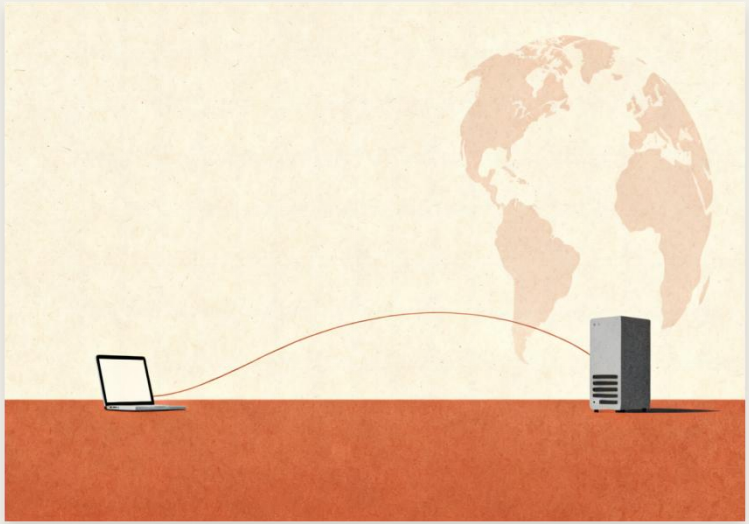
网络和安装是掉队高发区，我们把每一步的“为什么”都讲清楚。

第一步 · 打通网络

为什么这步绕不过去

Codex 每次干活，都要把你的指令发给 OpenAI 在海外的服务器，让最强的模型思考，再把结果传回。

而在中国大陆，默认网络访问不了 OpenAI。所以装 Codex 之前，必须先让电脑有一条能到达 OpenAI 的国际网络通道。



机制 · 为什么默认连不上

访问一个网址，其实是一串动作

① 问路（DNS 解析）	先问“chatgpt.com 的地址是多少”，可能被返回错误地址
② 拨号（连到那个 IP）	就算拿到地址，去连它的路可能被切断
③ 报出你要访问谁（TLS / SNI）	握手时明文暴露目标域名，可能据此触发拦截

任一环被卡，结果都是“打不开”。国际网络通道就是把这一整串请求保护起来、绕开拦截。

自己验证 · 别信断言

浏览器两步，确认真的连上

检查 A

打开 chatgpt.com

能正常加载登录页 = OpenAI 可达。

检查 B

打开 ipinfo.io

country 显示海外（非 CN）= 流量真的走了海外线路。

为什么查 ipinfo？它读取到达它的来源 IP 反查归属地——让第三方替你作证“流量从哪出去的”，不听客户端一面之词。

机制 · 一个高频坑

浏览器通了，App 却连不上?

根因：代理的“覆盖范围”。

只配浏览器（应用级代理）→ 只有浏览器走海外，Codex 走的还是默认线路。

系统代理 / TUN（全局）→ 整台电脑都走海外，Codex 才通。

一句话：让客户端“管住整台电脑”，而不只是浏览器。

第二步 · 装好 Codex Desktop

像装微信一样

下载 官方页面选你的系统（Mac / Windows）

安装 Mac 拖进“应用程序”，首次打开右键 → 打开

登录 用 ChatGPT 账号（各套餐都含 Codex）

选文件夹 新建一个空的练习文件夹，工作方式选 Local



机制 · 安装登录背后

两个你该懂的机制

机制拆解 · Mac 为什么拦？Gatekeeper

macOS 有个门卫，凡从网上下载的 App 默认先拦一道，看有没有经过苹果“公证”。

你右键 → 打开，等于告诉门卫“这个我认”，它就记住授权、以后不再拦。不是 Codex 有问题，是例行安检。

机制拆解 · 登录为什么弹浏览器？OAuth

你的密码只输在 OpenAI 官网，App 碰不到——更安全。官网发给 App 一枚“令牌”（入场凭证，不是密码），App 凭它干活。

这也解释了为什么偶尔要重新登录：令牌会过期。

它既是工作间，也是安全围栏

工作范围：

Codex 默认只读写这个文件夹里的东西。

沙箱围栏：

它想碰文件夹以外、或想联网，会被拦下问你。

所以用专门的空文件夹练习，不只是整洁——是把风险隔离在一处。最坏的情况，破坏也锁在这个文件夹里，波及不到你电脑上其他重要的东西。（完整的权限与还原，第四讲展开）

第三步 · 第一次对话

干活循环：五步一圈



不满意就继续说“哪里、改成什么”，再来一轮——这就是你日后干活的常态。

下一页看一段真实对话。

真实对话实录 · 从零建网页

你说需求 → 它给计划 + 预览

你：在这个目录做一个 index.html，页面居中显示“你好，我在学 Codex”，下面小字显示今天日期。纯前端、无依赖。

Codex：计划——新建 index.html，用原生 HTML + 内联 CSS 做垂直居中，日期用 JS 动态生成，不引入任何库。

改动预览 · index.html（新建）

```
+ <h1>你好，我在学 Codex</h1>

+ <p id="d"></p> … 〔批准〕〔拒绝〕
```

机制 · 改动预览的本质

它是一张“补丁”，分两步走

补丁（patch） = 一份精确到每一行的改动说明：“第几行，删掉这句、加上那句”。

① 生成补丁（提议）

你看到的“改动预览”就是这一步的产物。

② 应用补丁（落地）

你点批准，才执行这一步，改到真文件。

“生成”和“应用”分两步，正是你能先审后批的技术根源（呼应 Agent 循环的闸门）。

改一版：有红有绿

index.html（修改）

```
-   color:#1f2d3d;      （旧的那行）

+   color:#123a8b;      （改后的新行）

+   <p>继续加油</p>      （新增一行）
```

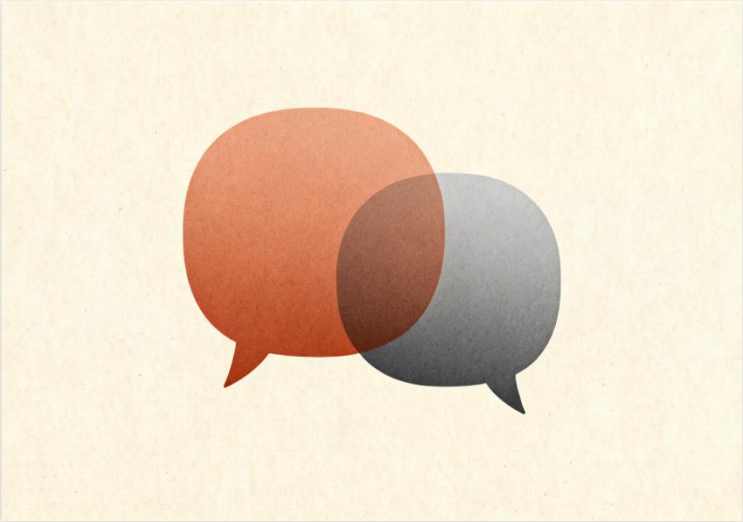
改一行 = 一条红（删旧）+ 一条绿（增新）。

改动越小越集中，通常越可控；一个小需求却给出一大片红绿，就要警惕它理解偏了。

为什么“说清楚”在底层有效

模型根据你给的文字，去预测“最合理的产出”。

指令模糊 → “合理产出”有成千上万种可能，它只能挑一个猜，多半不是你要的。



指令具体 → 你把可能性空间大幅收窄，命中率自然飙升。

prompting 的本质：用具体信息，一刀刀砍掉歧义，直到只剩你要的那一种。

好指令的核心

目标 + 边界 + 验收

目标

你到底要什么

做一个记录喝水量的网页

边界

范围 / 不要什么

只做单页，不要登录、不要联网

验收

怎么算做对了

点按钮数字加一，刷新不清零

这三样，眼熟吗？就是“先想清楚”落到指令上——不需要懂技术，靠你把事情想明白。

真实对话实录 · 对比

烂指令 vs 好指令

烂指令

你： 帮我做个喝水的网页

Codex： （只能猜）先按常见设想实现：含注册登录、云端存储、趋势图……

你： 并不想要这些——返工。

好指令

你： 单页 index.html：显示“今日 N 杯”+【加一杯】点一下加 1；数据存本地、不要登录/联网；点了加 1、刷新不清零。

Codex： （一次到位）单文件、纯前端，按验收标准做好。

进阶零件 · 按需叠加

让指令更准的五个“零件”

角色	“你是注重可读性的前端”——给它一个立场
背景	“给视力不好的长辈用”——它会自动调整（字大、按钮大）
示例	给一张截图 / 一段期望输出——一个例子胜过十句形容
分步	“先只做骨架给我看”——复杂活拆开，好把关
输出约定	“改动前先用中文列计划；只改 index.html”

机制：为什么“给例子”最有效？模型极擅长照葫芦画瓢——一个具体示例是高信息量锚点，几乎钉死方向。

思阳集团内部培训资料

机制 · 它的“记性”

上下文窗口：记忆是有限的

模型每次思考，只能“看到”一定量的文字（你的话 + 相关文件 + 之前的对话）。对话太长，较早的内容会被挤出去、被“遗忘”。

所以长对话里它会“忘掉”早先的约定——不是偷懒，是超出了窗口。

机制拆解 · 三个应对

① 重申——关键约定别指望它记一整场；

② 拆分——大活切成几段，别在一个对话里堆到爆；

③ 开新对话——它明显糊涂了，带上关键背景重新开始，往往比硬掰更快。

处理歧义 · 先对齐再动手

拿不准？逼它先复述

让它复述理解	“动手前，先用你的话复述一遍我的需求，我确认没歧义你再做。”
让它先列计划	“先列 3 步计划给我看，我点头再执行。”
给正反例	“我要的是 A 这种；不要 B 那种。”

迭代同理：不满意时不说“不好”，说“哪里、改成什么、怎样算好”。

今天的收获

你已经会了这些

✓ **看懂软件世界**	五角色 + 请求响应 + 本地/服务器 + 分层原理
✓ **理解 Codex 的机制**	大脑+手脚、Agent 循环、概率机器→别信断言
✓ **会装、会用**	网络分层、Gatekeeper/OAuth、沙箱围栏、干活循环
✓ **会好好说话**	目标+边界+验收、上下文窗口、先对齐再动手

门槛从来不在“会不会写代码”，而在会不会想清楚、指挥、验收、立规矩——而且你现在懂了为什么。

动手练习

做出你的“喝水计数器”网页



用今天学的“目标 + 边界 + 验收”，让 Codex 从零做出来：点按钮加一、刷新不清零。做完，用“别信断言”亲自打开验收。

下一讲 · 另一把利器 Claude